ModelBench: A Web-Based Platform for Empirical Machine Learning Model Evaluation and Comparative Performance Benchmarking

Mid-Semester Project Progress Report

Ujjwal Singh (Roll No: 262IS036) **Aryan** (Roll No: 262IS007)

M.Tech I Year (CSE-IS), Semester I

Course: CS801 Computing Lab | Course Instructor: Prof. P. Santhi Thilagam

Department of Computer Science and Engineering

National Institute of Technology Karnataka, Surathkal

Evaluation Date: October 12, 2026

1 Project Overview and Objectives

ModelBench is an interactive, web-based software platform designed for systematic training, evaluation, and side-by-side empirical comparison of classical Machine Learning (ML) algorithms. The project addresses a prevalent challenge in applied machine learning: experimentation across separate, ad-hoc Jupyter notebooks often leads to inconsistent train/test splits, hidden data leakage, and unstandardized metric comparisons. Furthermore, trained models typically remain trapped inside Python scripts, inaccessible to non-technical stakeholders.

ModelBench solves these challenges through a dual-persona design:

1. **End User Track:** Enables non-technical domain users to input familiar, human-readable features and receive live, formatted target predictions without requiring code execution or preprocessing expertise.
2. **ML Engineer / Researcher Track:** Provides a standardized experimentation harness to benchmark multiple algorithm families on identical, leak-free test partitions, analyze regularization hyperparameter sensitivity, and inspect diagnostic error plots.

The project is structured around an **80–90% ML core** supported by a lightweight, modular web integration layer (FastAPI backend and React 18 single-page dashboard).

2 Tasks Accomplished

As of the Mid-Semester milestone, the foundation of the ModelBench platform and the complete first working regression benchmark have been successfully implemented, validated, and integrated.

2.1 Data Ingestion and Exploratory Data Analysis (EDA)

We curated and analyzed the Medical Insurance Economics benchmark dataset ($N = 1,338$ records, 6 features, zero missing values):

- Statistical profiling revealed high positive skewness in the target variable (`charges`, range: $\$1,121$ to $\$63,770$, mean: $\$13,270$).
- EDA identified a critical non-linear interaction: smoking increases average annual medical expenses by $\approx 4\times$ ($\$32,050$ vs. $\$8,434$). More importantly, for smokers, a Body Mass Index (BMI) ≥ 30

(obesity) triggers an exponential cost surge, whereas non-smokers experience only a mild linear rise with BMI.

2.2 Leak-Free Preprocessing Architecture

To guarantee authentic out-of-sample evaluation without data leakage:

- Data is partitioned into an 80% training set (1,070 rows) and a 20% test set (268 rows, `random_state=42`) **prior** to any feature transformations.
- A Scikit-Learn `ColumnTransformer` is fit strictly on the training partition (X_{train}) and transforms the test partition (X_{test}).
- Numerical features (`age`, `bmi`, `children`) are normalized via `StandardScaler` (Z-score normalization: $\mu = 0, \sigma = 1$).
- Categorical features (`sex`, `smoker`, `region`) are encoded via `OneHotEncoder(drop='first')` to eliminate perfect multicollinearity (Dummy Variable Trap).

2.3 Model Exploration and Hyperparameter Tuning

Three regression model architectures were implemented and rigorously benchmarked:

1. **Ordinary Least Squares (OLS) Linear Regression:** Serves as the parametric baseline assuming independent additive contributions: $\hat{y} = \beta_0 + \sum \beta_j x_j$.

2. **2.4 Empirical Benchmarking Results**

All models were evaluated on the identical 268-sample test partition across Coefficient of Determination (R^2), Mean Absolute Error (MAE), and Root Mean Squared Error (RMSE):

Table 1: ModelBench Empirical Regression Benchmark Results (Test Partition $N = 268$)

Model Architecture	**Train R^2**	**Test R^2**	**Test MAE ($)**	**Test RMSE ($)**	**Evaluatio**
Linear Regression (OLS)	0.7417	0.7836	4,181.19	5,796.28	Baseline (Und
Ridge ($\alpha = 0.01$)	0.7417	0.7836	4,181.31	5,796.28	Identical
Ridge ($\alpha = 0.1$)	0.7417	0.7836	4,182.39	5,796.34	Stable
Ridge ($\alpha = 1.0$)	0.7417	0.7833	4,193.20	5,800.46	Moderate
Ridge ($\alpha = 10.0$)	0.7383	0.7791	4,300.89	5,856.32	Slight bia
Ridge ($\alpha = 100.0$)	0.6729	0.6859	5,217.80	6,997.74	Severe underfitt
Polynomial Ridge ($d = 2, \alpha = 10$)	**0.8407**	**0.8656**	**2,836.88**	**4,568.56**	**Optimal Perform**

Key Empirical Insight: Incorporating the non-linear interaction terms increased explained variance by +8.2% ($R^2 = 86.56\%$), reduced mean absolute error by $\$1,344.31$ (a 32% error reduction), and suppressed large residual penalties (RMSE reduced by $\$1,227.72$).

2.5 Diagnostic Error Analysis

Scatter plots of Actual vs. Predicted values and residual histograms revealed that OLS Linear Regression systematically underpredicts high-cost cases (heteroscedastic fan pattern). In contrast, Polynomial Ridge predictions tightly cluster along the ideal $y = x$ reference line with stabilized residual variance.

2.6 Modular Web Integration and Platform Architecture

- **ML Core (`ml_core/insurance_regression.py`):** Pure, reusable Scikit-Learn pipelines decoupled from web dependencies.

- **FastAPI Backend (`backend/main.py`):** Loads the dataset and fits pipelines once at server startup, enabling sub-millisecond inference (< 2 ms). Internal handling assigns baseline reference values for geographic region, presenting only 5 intuitive inputs (`age`, `sex`, `bmi`, `children`, `smoker`) to end users.
- **React Frontend (`frontend/src/App.jsx`):** A clean 3-tab workbench supporting: (1) Live Model Prediction, (2) Model Comparison Scorecard, and (3) Dataset Insights & Diagnostic Scatter Plots.

3 Tasks Yet to Be Completed

For the second half of the semester (Continuous Evaluation 2 and End-Semester Evaluation), the following modules will be developed:

3.1 Phase 2: Classification Benchmark Engine (CE-2 Milestone)

(a) **Dataset Acquisition and Preprocessing:** Ingestion of a real-world classification dataset (e.g., Telco Customer Churn, $N \approx 7,000$ or Bank Loan Default Risk). Implementation of class imbalance inspection and stratified splitting.

(b) **Algorithm Implementation:**

- Parametric Baseline: Logistic Regression (L_1 Lasso and L_2 Ridge penalties).
- Non-Parametric: K-Nearest Neighbors (KNN with distance metric tuning).
- Tree-Based Models: Decision Tree Classifier (pruning via `max_depth` and `min_samples_split`).
- Ensemble Architectures: Random Forest Classifier (bootstrap aggregation and feature subspace sampling).
- Margin-Based: Support Vector Classifier (Linear and Radial Basis Function RBF kernels).

(c) **Classification Evaluation Metrics:** Precision, Recall, F_1-Score, Specificity, Accuracy, and Area Under the ROC Curve (ROC-AUC).

(d) **Diagnostic Visualizers:** Interactive Confusion Matrix heatmaps and Receiver Operating Characteristic (ROC) curve overlays.

3.2 Phase 3: Automated Hyperparameter Optimization and UI Extensibility (EndSem Milestone)

(a) **Cross-Validated Search Module:** Integrating k-fold cross-validation with automated `GridSearchCV` and `RandomizedSearchCV` to display hyperparameter sensitivity surfaces.

(b) **Multi-Dataset UI Switching:** Upgrading the React frontend with schema-driven dataset selectors, dynamically toggling table columns between regression metrics (R^2, MAE) and classification metrics (F_1, AUC).

(c) **Comprehensive Final Reporting and Verification:** Final benchmarking report comparing classical models against tree ensembles across computational latency, interpretability, and generalization.

4 Team Responsibilities and Timeline

4.1 Division of Work

- **Ujjwal Singh (262IS036):** Lead on Exploratory Data Analysis, non-linear interaction discovery, Polynomial Feature modeling, Decision Tree and Random Forest implementations, Confusion Matrix diagnostics, and Master Jupyter Notebook maintenance.

- **Aryan (262IS007):** Lead on leak-free `ColumnTransformer` pipeline architecture, Ridge regularization experiments, Logistic Regression and SVM implementations, FastAPI backend REST integration, and dynamic schema coordination.

4.2 Milestone Schedule

Table 2: ModelBench Project Execution Timeline

Milestone	**Target Due Date**	**Key Deliverables**
Phase 1: Extended Abstract	September 04, 2026	Problem definition, literature review, LaTeX abstract [Com
Phase 2: Continuous Eval 1	September 21, 2026	Regression pipeline, leak-free ColumnTransformer, CE-1 PP
Phase 3: MidSem Evaluation	**October 12, 2026**	**Live code demonstration, MidSem LaTeX report, P**
Phase 4: Continuous Eval 2	November 01, 2026	Classification engine (Churn/Credit), Decision Trees, Rando
Phase 5: EndSem Evaluation	November 10–13, 2026	Automated GridSearch, multi-dataset UI, comprehensive fi

5 Conclusion

At the Mid-Semester stage, ModelBench successfully fulfills its defined core objectives for regression experimentation. It demonstrates empirical performance gains ($+8.2\% R^2$) through leak-free polynomial interaction modeling, connects the underlying Scikit-Learn pipelines to a sub-millisecond REST API, and provides an accessible user interface for both domain users and ML researchers. The scheduled roadmap for CE-2 and EndSem will expand the platform into a comprehensive, multi-task benchmark suite.